Project 20.1 — ShelfQuery-Library-Catalog-Q-And-A-In-Plain-English

A practice twin of Project 20 — SQLSage. Same skills — turn a plain-English question into a single read-only `SELECT`, **guard** it so nothing but a safe query ever runs, execute it against SQLite, and phrase the rows in one sentence — on a new real-world job. **There are no steps here on purpose.** Read the story, build it yourself, and open the hints or solution only if you get stuck.

Type: NL-to-SQL analytics agent · **Stack:** Python, LangChain (structured output), SQLite (`sqlite3`), OpenAI SDK, Pydantic · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A librarian wants quick answers from the catalog — "how many books are on loan?", "which books by Le Guin do we have?" — but they don't write SQL, and the catalog is the library's system of record: a bot must **never** change it, only read it.

You are going to build **ShelfQuery**: ask the catalog a question in plain English, get an answer — and a hard guarantee that only read-only queries ever run.

- The catalog **schema** (the `CREATE TABLE` statements) is read and handed to the model.
- The model returns a single read-only `SELECT` as **structured output**.
- A deterministic **guard** rejects anything that isn't a safe single `SELECT` — no `INSERT`, `UPDATE`, `DELETE`, `DROP`, or chained statements.
- The **runner** executes the query; the **answer** step phrases the rows in one sentence.

The one lesson of this project: same as SQLSage — letting a model write SQL is powerful and dangerous, and the safety is a **deterministic guard**, not a hope. The guard (SQL in, safe/unsafe out) and the runner (query in, rows out) need no model, so you test them with hand-written SQL and a tiny in-memory database. The model only writes the query and phrases the result.

How it works, from a question to an answer (picture the whole machine)

A question comes in. `schema_text` reads the catalog's `CREATE TABLE` statements and hands them to the model, which returns a single `SELECT` as structured output. Before that SQL runs, `is_safe_select` checks it — must start with `SELECT`, no write or schema keyword, a single statement. Only then does `run_select` execute it and return the rows, and `summarize_answer` turns those rows into one plain sentence.

That's the whole machine: **question → generate SELECT → guard → run → one-sentence answer.**

What you are building (the pieces)

- A sample catalog database: `build_sample_db(db_path)` (books + loans), `schema_text(db_path)`, `run_select(db_path, sql)`.
- A deterministic `is_safe_select(sql)` guard — single read-only `SELECT` only.
- A `SQLQuery` shape and a `generate_sql(model, schema, question)` (structured output).
- A grounded `summarize_answer(model, question, rows)`.
- A pipeline `ask(model, db_path, question)` that guards the SQL before running it.

Inputs: a plain-English question. **Outputs:** a one-sentence answer (and the SQL that ran). **Rules:** only a single read-only `SELECT` ever runs; an unsafe generated query raises `UnsafeQueryError` and never touches the database; the answer is phrased from the returned rows. The guard and runner are plain Python, testable with no model.

A real example to build and test against

- A tiny catalog: `books(id, title, author, available)` and `loans(id, book_id, member, returned)`.
- "How many books are currently on loan?" → `SELECT count(*) FROM loans WHERE returned = 0` → "There are 2 books on loan."
- A generated query like `DELETE FROM books` → the guard rejects it, `UnsafeQueryError` is raised, and the catalog is untouched.

How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 20 / Doc05 structured output if hazy.)
2. Build the **deterministic parts first** — `is_safe_select` and the database (`build_sample_db`, `schema_text`, `run_select`) — and test them with hand-written SQL and a temp database, no model.
3. Build the structured `generate_sql` and the grounded `summarize_answer`.

4. Wire the pipeline: schema → generate → **guard** → run → summarize.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

"Let a non-technical person query our database in plain English — safely" is the shape of internal analytics bots, admin dashboards, and support tools alike. The skill is the **guard**: an LLM can generate anything, so a deterministic check, not a hope, is what decides whether a query runs. Build ShelfQuery and you can put a natural-language front end on any read-only database without risking the data behind it.

Done when

- The question becomes a single `SELECT` via structured output.
- The guard rejects anything that isn't a safe single read-only `SELECT`.
- An unsafe generated query raises `UnsafeQueryError` and never runs.
- The answer is phrased from the returned rows, not invented.
- Your tests pass: the guard and database (hand-written SQL, no model) and a live question answered with the right number.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.